

Computer Vision, Lezione 7

Laboratorio: Fourier sulle immagini e classificazione con banchi di filtri

Appunti ricavati dai notebook in Lessons/L7/lab/ (demo, filter_bank, 06-image-blending)

Come leggere questi appunti

La lezione 4 faceva spettro e convoluzione su segnali 1D (audio). Questo laboratorio fa **la stessa cosa in 2D, sulle immagini**, e poi usa le convoluzioni come estrattori di feature per un classificatore.

Parte	Notebook	Domanda a cui risponde
1	demo.ipynb	Com'è fatto lo spettro di un'immagine e come si filtra?
2	filter_bank.ipynb	Quanto si classifica bene con filtri fissi scelti a mano?
3	06-image-blending	Identico a quello della lezione 6: vedi sezione 3.

Teoria collegata: convoluzione e teorema di convoluzione (lezione 3), piramidi e aliasing (lezione 6), FFT 1D (laboratorio 4).

Riquadri: **blu** = il concetto in due righe, **rosso** = punti in cui il notebook sbaglia o confonde, **verde** = cosa ricordare, **grigio** = soluzione di un esercizio, eseguita.

Tre parole da sapere prima di iniziare

- **Frequenza spaziale**: quante volte un motivo si ripete per pixel, in *cicli/pixel*. Va da -0.5 a 0.5 (Nyquist). Una frequenza 2D è una coppia (f_x, f_y) .
- **Spettro 2D**: la FFT dell'immagine (`np.fft.fft2`). Si disegna il modulo in scala log, con la frequenza zero al centro (`fftshift`).
- **Banco di filtri**: una lista di kernel. Ogni kernel convoluto con l'immagine dà una mappa di risposta; le mappe, ridotte, diventano le feature del classificatore.

1 Fourier e convoluzione sulle immagini (demo.ipynb)

Immagine di lavoro: camera di scikit-image, ritagliata a 256×256 ([96:352, 128:384]).

1.1 Coordinate di Fourier: i fatti da sapere

Cosa	Come / quanto vale (rieseguito)
<code>np.fft.fft2(img)</code> <code>np.fft.fftfreq(W)</code>	FFT sulle due assi, uscita $H \times W$ complessa frequenze in cicli/pixel, <i>nell'ordine della FFT</i> : $0, 1/W, \dots$, poi le negative
<code>F[0,0] / (H*W)</code>	media dell'immagine (componente continua, DC): 0.3984
Parseval: $\sum f ^2 = \sum F ^2 / (HW)$	$16143.14 = 16143.14$
<code>ifft2(fft2(img)).real</code> <code>fftshift / ifftshift</code>	ritorna l'immagine, errore massimo $5.6 \cdot 10^{-16}$ porta DC al centro per disegnare; <code>ifftshift</code> pri- ma di invertire uno spettro centrato

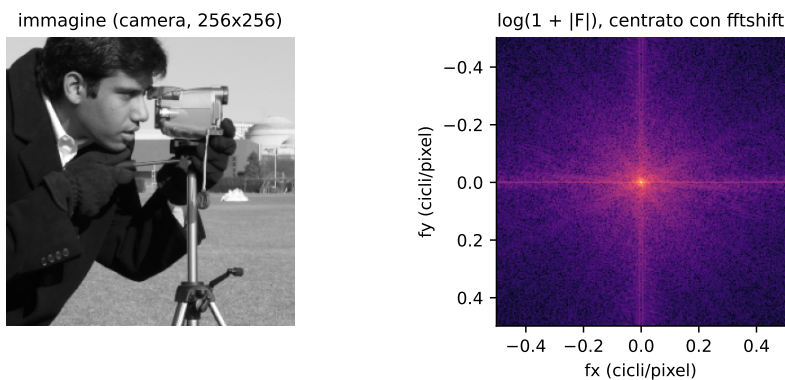


Figura 1: Lo spettro di un'immagine naturale: quasi tutta l'energia vicino al centro (basse frequenze). La croce orizzontale e verticale viene dai bordi dell'immagine: la FFT la tratta come periodica e il salto tra lato destro e sinistro è un bordo netto.

1.2 Onde semplici: prevedere lo spettro

Un'onda $g[y, x] = \cos(2\pi(k_x x/W + k_y y/H))$ con k_x, k_y interi ha **due picchi** in $\pm(k_y, k_x)$, ognuno di modulo $HW/2$ (qui 32768, verificato).

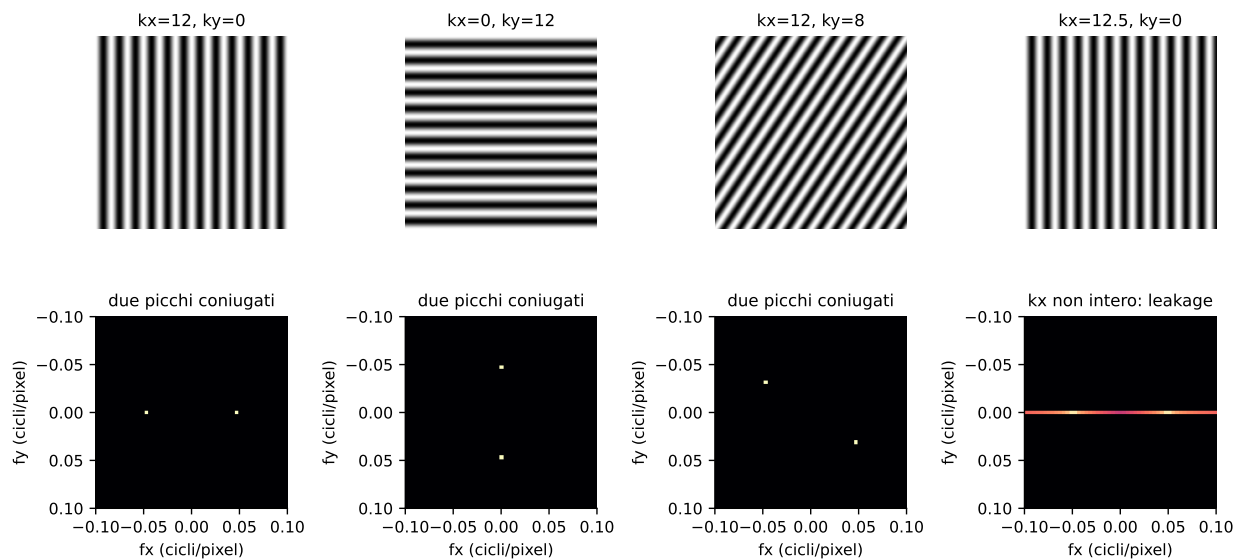


Figura 2: Zoom sulla zona $|f| < 0.1$. Le strisce verticali ($k_x = 12$) danno picchi sull'asse orizzontale: il vettore $(k_x/W, k_y/H)$ è *perpendicolare* alle strisce. A destra $k_x = 12.5$ non è intero: l'onda non si chiude sul bordo e l'energia si spande su tutta la riga (spectral leakage): 256 bin sopra l'1% del massimo, contro 2.

Risposte agli esercizi semplici della cella 5:

- Cambiando (k_x, k_y) : la **direzione** dei picchi è perpendicolare alle strisce, la **distanza** dal centro è la frequenza (strisce più fitte = picchi più lontani).
- Frequenza non intera: leakage, come in figura.
- Somma di onde: lo spettro è la somma degli spettri (la FFT è lineare), quindi una coppia di picchi per ogni onda.

1.3 Modulo, fase e traslazione

Spostare l'immagine **cambia solo la fase** dello spettro, non il modulo:

$$\text{shift di } (d_y, d_x) \iff F \cdot e^{-2\pi i(f_y d_y + f_x d_x)}$$

Il notebook lo prova con `np.roll(image, (27, -19))`: la ricostruzione con la rampa di fase coincide con l'immagine spostata e il modulo della FFT non cambia (entrambe le asserzioni passano). Vale per lo spostamento *circolare* (`np.roll`): quello che esce da un lato rientra dall'altro.

L'idea in breve

Il modulo dice *quali* frequenze ci sono, la fase dice *dove* stanno. Uno spostamento non cambia il contenuto in frequenza, quindi non tocca il modulo.

1.4 Bordi: cosa c'è fuori dall'immagine

`scipy.signal.convolve2d(img, k, mode="same", boundary=...)`. Il parametro `mode` decide solo la *dimensione* dell'uscita; `boundary` decide come si estende l'immagine.

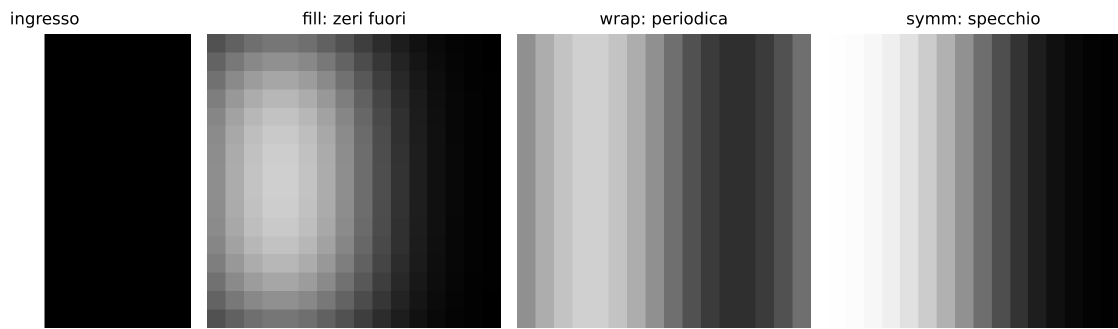


Figura 3: Quadrato 16×16 metà bianco, sfocato con una gaussiana $\sigma = 3$.

boundary	Estensione	angolo bianco (0,0)	bordo nero (8,15)
fill	zeri fuori: i bordi si scuriscono	0.318	0.005
wrap	periodica: il lato destro (nero) entra a sinistra	0.563	0.437
symm	specchio, ripetendo il pixel di bordo	0.993	0.007

Verificato: `symm` equivale a `np.pad(..., mode="symmetric")`, cioè $[1, 2, 3] \rightarrow [\dots 2, 1, 1, 2, 3, 3, 2 \dots]$ (il pixel di bordo è ripetuto; `reflect` invece non lo ripete).

Collegamento importante: filtrare moltiplicando nello spettro (`fft2`, `maschera`, `ifft2`) equivale a una convoluzione con `boundary="wrap"`, perché la DFT tratta l'immagine come periodica.

1.5 Passa-basso ideale e gaussiano

Le due maschere del notebook, con $r = \sqrt{f_x^2 + f_y^2}$:

- **ideale:** $M = 1$ se $r \leq 0.08$, altrimenti 0 (tiene il 2.0% dei coefficienti);
- **gaussiana:** $M = \exp(-2\pi^2\sigma^2r^2)$ con $\sigma = 2$. È la trasformata di un kernel gaussiano spaziale di deviazione standard σ : verificato, il kernel che corrisponde alla maschera ha std 2.000 pixel.

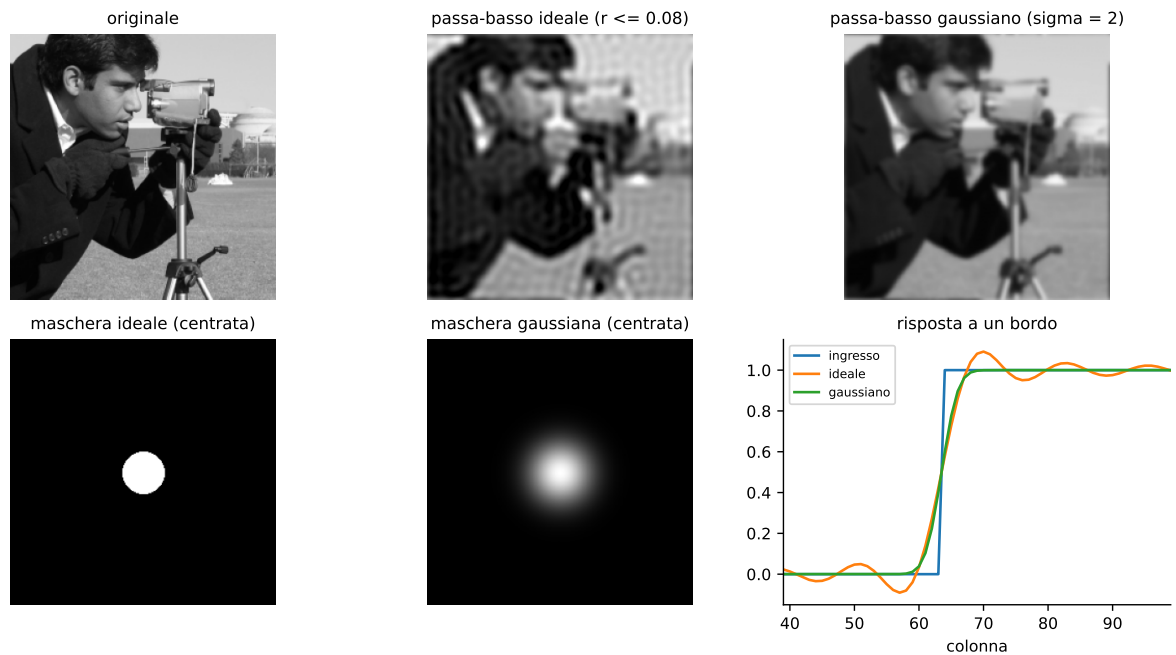


Figura 4: Il passa-basso ideale produce aloni attorno ai contorni (ringing). Sul bordo netto (in basso a destra): l'ideale oscilla tra -0.091 e 1.091 (9% oltre i valori reali), il gaussiano resta in $[0, 1]$.

L'idea in breve

Taglio netto in frequenza = sinc nello spazio = oscillazioni (ringing). La gaussiana non ha taglio netto: nello spazio è ancora una gaussiana, sempre positiva, quindi niente oscillazioni. Stesso fenomeno visto nell'audio in L4.

1.6 Passa-alto come residuo

Con la maschera complementare $1 - M$: $f_{\text{alto}} = f - f_{\text{basso}}$. Verificato: alto + basso = immagine, e la media del passa-alto è 0 (la DC è tolta).



Figura 5: Il residuo alto contiene bordi, tessiture fini e rumore insieme: un passa-basso non può togliere il rumore senza togliere anche i dettagli.

1.7 Togliere un'interferenza periodica (filtro notch)

Se il disturbo è un'onda a frequenza nota, nello spettro è **una coppia di picchi**. Si azzerano solo quelli, con una maschera *notch* ("tacca"): 1 ovunque tranne due piccole gaussiane rovesciate sui due picchi coniugati.

```
mask *= 1 - np.exp(-d2 / (2*width**2)) # per ognuno dei due picchi, width = 1/W
```

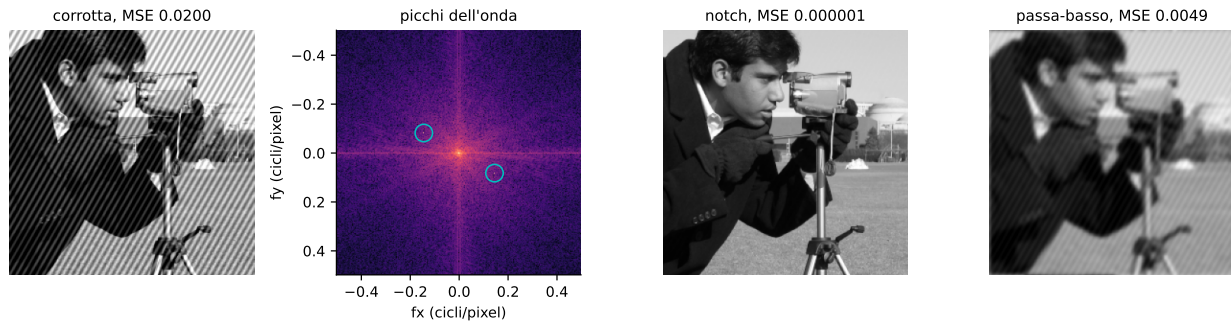


Figura 6: Onda di ampiezza 0.20 a $(k_x, k_y) = (37, 21)$ sommata a camera.

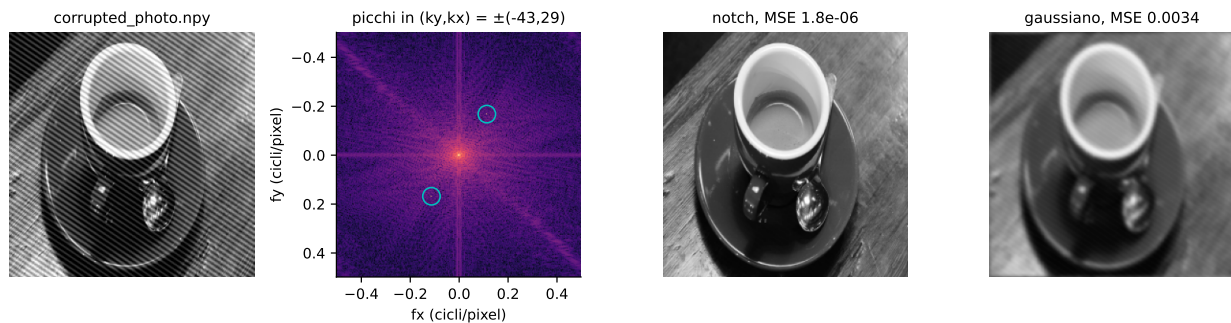
Immagine	MSE rispetto all'originale (rieseguito = salvato)
Corrotta	0.020000
Notch	0.000001
Passa-basso gaussiano	0.004886

0.02 non è un caso: la media di $(A \cos)^2$ è $A^2/2 = 0.2^2/2 = 0.02$. Il notch toglie quasi solo il disturbo; il passa-basso toglie il disturbo ma sfoca l'immagine.

1.8 Esercizio 1: la foto corrotta (`corrupted_photo.npy`)

Soluzione (eseguita)

1. **Trovare i picchi:** si calcola `abs(fft2(obs))`, si ignora il centro ($r < 0.05$, dove c'è l'immagine) e si prende il massimo. Risultato: una sola coppia, $(k_y, k_x) = \pm(-43, 29)$, circa 400 volte sopra i vicini. Ampiezza dell'onda $2|F|/(HW) = 0.181$.
2. **Maschera:** `notch_mask(obs.shape, 29/256, -43/256, width=1/256)`, la stessa funzione del notebook.
3. **Ricostruire:** `ifft2(Fo * notch).real`.
4. **Confronto:** l'immagine pulita è `data.coffee()` di scikit-image, in grigio e ridotta a 256×256 (trovata confrontando con le immagini di esempio della libreria). Rispetto a quella: osservata MSE 0.0162, notch $1.8 \cdot 10^{-6}$, gaussiano ($\sigma = 2$) 0.0034.



1.9 Esercizio 2: compressione con Fourier (Fig. 16.8 del visionbook)

Soluzione (eseguita)

Si tengono solo i coefficienti di modulo più grande e si azzerano gli altri:

```
thr = np.sort(np.abs(F).ravel())[:-1][int(keep*F.size) - 1]
rec = np.fft.ifft2(np.where(np.abs(F) >= thr, F, 0)).real
```

I due coefficienti coniugati hanno lo stesso modulo, quindi vengono tenuti o buttati insieme e la ricostruzione resta reale.

Coefficienti tenuti	quanti	MSE	PSNR
20%	13107	0.00068	31.7 dB
5%	3276	0.00267	25.8 dB
1%	655	0.00776	21.1 dB
0.2%	131	0.01674	17.8 dB



Figura 7: Con il 5% dei coefficienti l'immagine è ancora riconoscibile: l'energia è concentrata in pochi coefficienti a bassa frequenza. Scendendo, compaiono sfocatura e ringing.

Attenzione: il notebook sbaglia o confonde

- Figura vuota.** La cella 2 chiama `plt.figure()` prima di `plt.matshow(FY)`: `matshow` crea già la sua figura, quindi resta una figura vuota (nell'output salvato: `<Figure size 704x528 with 0 Axes>`).
- "Shift" vuol dire shift circolare.** La proprietà "lo spostamento cambia solo la fase" vale esatta per `np.roll`. Per uno spostamento vero (con pixel nuovi che entrano dal bordo) il modulo cambia un po'.
- Numeri diversi di poco.** Errore di ricostruzione salvato $6.66 \cdot 10^{-16}$, rieseguito $5.55 \cdot 10^{-16}$: dipende dalla versione di NumPy, non è un errore.

Da ricordare per l'esame

- Onda $\Rightarrow$ coppia di picchi coniugati, perpendicolari alle strisce. Frequenza non intera $\Rightarrow$ leakage.
- Traslazione = rampa di fase; il modulo non cambia.
- Filtrare con la FFT = convoluzione circolare (wrap).
- Passa-basso ideale $\Rightarrow$ ringing (overshoot del 9%); gaussiano no.
- Interferenza periodica: notch sui due picchi, molto meglio di un passa-basso (MSE 10^{-6} contro $5 \cdot 10^{-3}$).

2 Classificare con banchi di filtri fissi (`filter_bank.ipynb`)

L'idea in breve

Invece di imparare i filtri (come una CNN), li si sceglie a mano: si convolve l'immagine con ogni filtro, si applica una non linearità, si riduce con il pooling e si dà tutto a un classificatore *lineare*. Il laboratorio misura quanto conta ogni scelta: quali filtri, quale non linearità, quanto pooling.

2.1 Pipeline e dati

$$x \longrightarrow x * w_k \longrightarrow \rho(x * w_k) \longrightarrow P(\rho(x * w_k)) \longrightarrow \text{classificatore lineare}$$

Passo	Nel codice
Convoluzione	<code>signal.fftconvolve(..., mode="same")</code> , zero padding
Non linearità ρ	<code>none</code> , <code>abs r </code> , <code>square r^2</code> , <code>rectify $\max(r, 0)$</code>
Pooling P	blocchi $p \times p$ non sovrapposti, media (avg) o massimo (max)
power	$z \leftarrow \text{sign}(z) z ^{\text{power}}$, opzionale
Classificatore	<code>StandardScaler</code> + <code>RidgeClassifier(alpha=1)</code> , fisso

- **Dati:** Fashion-MNIST, immagini 28×28 di vestiti in 10 classi. Train 20000, validazione 10000 (dal training set ufficiale, senza sovrapposizione), test 10000. Il notebook li scarica da solo in `lab/data` (30 MB, già scaricati).
- **Numero di feature** = $\text{filtri} \times (28/p)^2$. Con pool 4: 49 per filtro.
- Si confrontano le scelte sulla **validazione**; il test si usa solo alla fine.

L'idea in breve

Perché serve la non linearità. Convoluzione e pooling medio sono lineari: senza ρ le feature sono combinazioni lineari dei pixel, e un classificatore lineare su di esse non può fare meglio che sui pixel. Verificato: 7 filtri, `none`, pool 1 (5488 feature) $\rightarrow$ 0.8127, pixel grezzi 0.8129.

2.2 Come leggere i numeri

Le soluzioni degli esercizi sono state tolte dal notebook (`assert False`), ma sono rimasti gli output del docente. Ho ricostruito le soluzioni e rieseguito tutto. Per tarare il confronto: le celle con codice completo (Gabor) danno valori diversi al massimo di 0.0004 da quelli salvati, per le versioni diverse delle librerie. Differenze più grandi indicano che il docente ha usato un filtro un po' diverso.

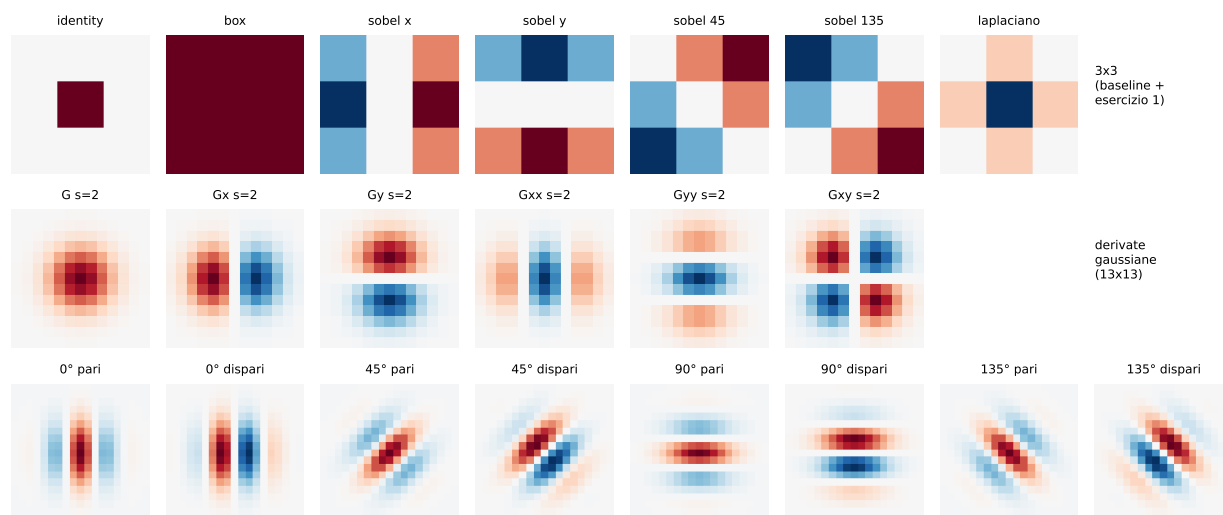


Figura 8: I filtri usati. Rosso positivo, blu negativo. Riga 3: Gabor a $\sigma = 3$, $\lambda = 8$, 4 orientazioni, fase pari (coseno) e dispari (seno).

2.3 Pixel e baseline (identity, box, Sobel)

Soluzione (eseguita)

```
box = np.ones((3, 3)) / 9
sobel_x = np.array([[ -1,  0,  1], [ -2,  0,  2], [ -1,  0,  1]], float)
sobel_y = sobel_x.T
```

Configurazione	feature	salvato	rieseguito
Pixel grezzi (identity, none, pool 1)	784	0.8128	0.8129
Baseline (identity, box, Sobel x/y; abs, pool 4)	196	0.8376	0.8381
Baseline senza identity	147		0.8343
Solo identity, abs, pool 4	49		0.7498

QB.1 Con identity 0.8381, senza 0.8343: il pixel grezzo aiuta poco. La baseline batte i pixel con 4 volte meno feature: il pooling riassume ogni blocco 4×4 ed è tollerante a piccoli spostamenti; da solo però perde troppo (identity + pool 4: 0.7498). Servono i filtri di bordo prima del pooling.

QB.2 (risposta nel notebook) Sobel x risponde ai bordi verticali: pantaloni, vestiti, borse. Sobel y ai bordi orizzontali: cinghie dei sandali, orli e maniche. Il rapporto tra le due risposte in ogni blocco è un descrittore grezzo dell'orientazione.

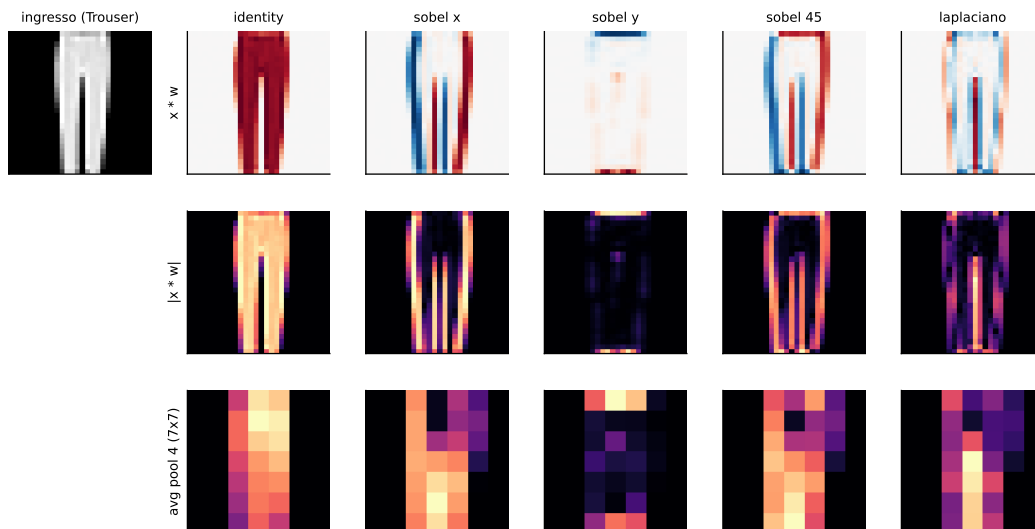


Figura 9: Un pantalone attraverso la pipeline. Sobel x vede le gambe, Sobel y quasi niente. L'ultima riga ($7 \times 7 = 49$ numeri per filtro) è quello che vede il classificatore.

2.4 Esercizio: diagonali e laplaciano

Soluzione (eseguita)

```
sobel_d45 = np.array([[0, 1, 2], [-1, 0, 1], [-2, -1, 0]], float)
sobel_d135 = np.array([[2, -1, 0], [-1, 0, 1], [0, 1, 2]], float)
laplacian = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], float) # 4-vicini
```

Stessa idea di Sobel: derivata attraverso il bordo, media lungo il bordo, ruotata di 45 gradi.

Banco	feature	salvato	rieseguito
baseline + diagonali	294	0.8573	0.8600
baseline + laplaciano	245	0.8456	0.8484 (4-vicini) / 0.8460 (8-vicini)
baseline + diagonali + laplaciano	343	0.8638	0.8641

Risposta: le diagonali aiutano di più (+2.2 punti contro +1.0 del laplaciano): aggiungono orientazioni che Sobel x/y non coprono. Il laplaciano è isotropo, quindi non aggiunge informazione di orientazione.

2.5 Esercizio: derivate gaussiane a più scale

Soluzione (eseguita)

Per ogni σ : la gaussiana e le 5 derivate già implementate (x, y, xx, yy, xy), più una identity in testa. Con una scala sono 7 filtri = 343 feature, lo stesso numero dell'output salvato.

```
gd_bank = [identity] + [k for s in sigmas for k in
    [gaussian_kernel(s)] + [gaussian_derivative_kernel(s, o)
        for o in ["x", "y", "xx", "yy", "xy"]]]
```

Scale σ	feature	salvato	rieseguito
1	343	0.8583	0.8634
2	343	0.8562	0.8605
1, 2	637	0.8748	0.8809
0.7, 1, 2, 3	1225	0.8865	0.8905

Una scala sola vale come i 7 filtri 3×3 ; **più scale insieme** aiutano (+2.7 punti rispetto a $\sigma = 1$), perché vedono sia i dettagli fini sia le strutture larghe (spazio tra le gambe, colletto). Ipotesi sulla differenza di circa 0.5 punti dai valori salvati: il docente ha usato un banco di 7 filtri un po' diverso (non ricostruibile dagli output).

2.6 Filtri di Gabor

Un filtro di Gabor è una gaussiana moltiplicata per un'onda: risponde a una *posizione*, una *orientazione* θ e una *frequenza* $1/\lambda$. Somigliano alle cellule semplici della corteccia visiva (V1) e ai filtri del primo strato delle CNN. Codice già completo nel notebook.

Orientazioni (x 2 scale x 2 fasi, + identity)	feature	salvato	rieseguito
2	441	0.8673	0.8675
4	833	0.8865	0.8867
8	1617	0.8949	0.8945

Più orientazioni = più accuratezza, con guadagno che cala (2→4: +1.9 punti, 4→8: +0.8).

2.7 Esercizio: non linearità e pooling

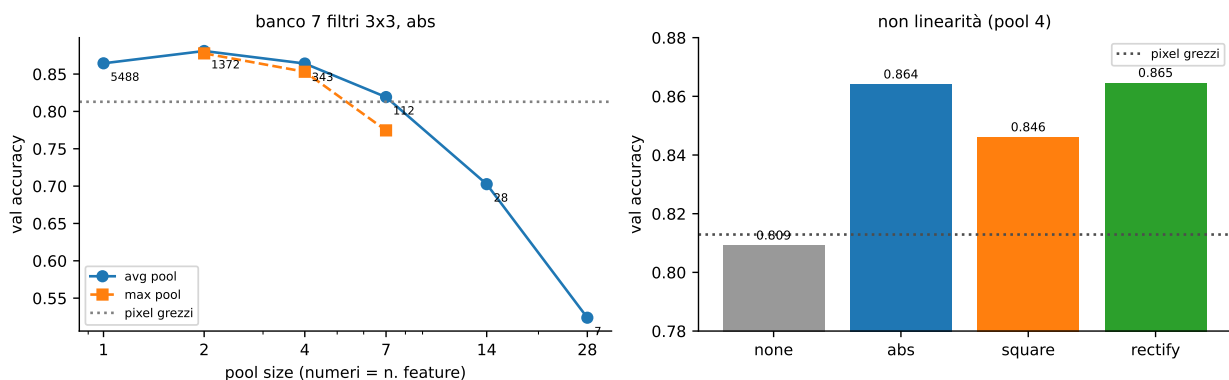


Figura 10: Banco a 7 filtri 3×3 (baseline + diagonali + laplaciano), rieseguito.

Non linearità (pool 4)	val	Pooling (abs, media)	feature	val
none	0.8092	1	5488	0.8644
abs	0.8641	2	1372	0.8809
square	0.8461	4	343	0.8641
rectify	0.8646	7	112	0.8192
abs + power=0.5	0.8715	14 / 28	28 / 7	0.7026 / 0.5238

Max pooling (abs): 0.8777 (pool 2), 0.8531 (pool 4), 0.7746 (pool 7), sempre sotto la media.

Risposte alle domande:

- **Senza non linearità:** verificato, non si supera il pixel grezzo (0.8127 con pool 1, 0.8092 con pool 4, contro 0.8129).
- **Pooling:** si guadagna tolleranza ai piccoli spostamenti e meno feature (meno rischio di overfitting, più veloce); si perde la posizione. Il migliore qui è pool 2; oltre 4 si perde troppo (pool 28 = un numero per filtro: 0.52).
- **power=0.5** (+0.7 punti): comprime i valori grandi, così poche risposte fortissime non dominano il classificatore lineare. Spiegazione standard, non misurata qui.
- square va peggio di abs: eleva al quadrato proprio i valori grandi, l'effetto opposto di power=0.5.

2.8 Esercizio: filtri casuali

Soluzione (eseguita)

```
rng = np.random.default_rng(0)
def random_bank(n, size=5):
    return [rng.standard_normal((size, size)) for _ in range(n)]
```

Filtri casuali 5×5	feature	salvato	rieseguito (seed 0)	banco a mano con stesse feature
4	196	0.8349	0.8361	baseline 0.8381
13	637	0.8714	0.8774	derivate gauss. $\sigma = 1, 2$: 0.8809
34	1666	0.8922	0.8925	Gabor 8: 0.8945

Q6.1: con abbastanza filtri, i casuali arrivano quasi ai banchi fatti a mano. **Q6.2:** i casuali non richiedono progettazione ma non si sa cosa misurano (poco interpretabili) e servono più filtri per lo stesso risultato; quelli a mano sono interpretabili ("bordo a 45 gradi") e più efficienti a parità di feature.

2.9 Challenge e test finale

Output salvati del docente (composizione esatta dei banchi non nota, quindi non rieseguiti):

Configurazione del docente	feature	val
energia di Gabor 8x2 ("complex cells"), power 0.5	882	0.8884
Gabor 8, power 0.5	1666	0.9002
Gabor 8x3 + derivate gaussiane, power 0.5	3038	0.9094

L'energia di Gabor è $\sqrt{r_{\text{pari}}^2 + r_{\text{dispari}}^2}$: la risposta non dipende dalla fase, come nelle cellule complesse di V1.

Test (riallenando su train + validazione), rieseguito:

Configurazione	test accuracy
Pixel grezzi	0.8101
Baseline	0.8383
Baseline, power 0.5	0.8557
Gabor 8 + identity, power 0.5 (val 0.9055)	0.8983

Attenzione: il notebook sbaglia o confonde

Numeri di cella = numerazione del notebook originale.

1. **'ReLU' non esiste.** Il testo (cella 5) chiama la non linearità 'ReLU', ma nel dizionario NONLINEARITIES la chiave è 'rectify': nonlinearity="ReLU" dà KeyError.
2. **Il banco finale non è il migliore.** Cella 32: FINAL_BANK = baseline_bank # best validation entry. Il commento chiede di mettere il banco migliore, il codice mette la baseline. Va sostituito con il proprio banco migliore.
3. **Output da esecuzioni diverse.** La cella 32 ha salvato un NameError (baseline_bank non definito), ma la cella 34, che usa le stesse variabili, ha un grafico: gli output salvati non vengono da un'unica esecuzione dall'alto in basso.
4. **show_responses usa sempre pool 4.** Disegna `pool(..., 4).reshape(7, 7)` qualunque sia il `pool_size` provato: le mappe mostrate non cambiano se si cambia il pooling.

Da ricordare per l'esame

- Filtri lineari + classificatore lineare = classificatore lineare sui pixel. Serve una non linearità ($|\cdot|$, ReLU).
- Pooling: tolleranza a piccoli spostamenti e meno feature, al prezzo della posizione. C'è un punto ottimo (qui pool 2).
- Più orientazioni e più scale aiutano. Filtri casuali in numero sufficiente arrivano vicino a quelli a mano.
- Manca la gerarchia (filtri su filtri) e l'apprendimento dei filtri: è quello che aggiunge una CNN.

3 Image blending (06-image-blending)

Il notebook in lab/06-image-blending è **identico byte per byte** a quello della lezione 6 (stesso hash MD5). È già spiegato in Appunti/CV_L6_Aliasing_Scale_Invariance_appunti.txt, parte B, con i due bug verificati e la figura L6_blending_confronto.png.

In breve: piramide laplaciana di mela e arancia, piramide gaussiana della maschera, somma pesata livello per livello, ricostruzione (multi-band blending, Burt e Adelson 1983). Le bande basse si mescolano su una zona larga, quelle alte su una stretta: niente cucitura visibile e niente effetto "fantasma".

4 Domande tipo esame

1. **Dove sono i picchi dello spettro di $\cos(2\pi(k_x x/W + k_y y/H))$?**
In $\pm(k_y, k_x)$, di modulo $HW/2$. La direzione è perpendicolare alle strisce.
2. **Cosa succede allo spettro se spostato l'immagine?**
Il modulo resta uguale, la fase si moltiplica per $e^{-2\pi i(f_y d_y + f_x d_x)}$ (spostamento circolare).

3. **Perché il passa-basso ideale dà aloni e il gaussiano no?**
Il taglio netto corrisponde a una sinc nello spazio, che oscilla (overshoot del 9% su un bordo). La gaussiana resta gaussiana, positiva, senza oscillazioni.
4. **Come si toglie un disturbo periodico?**
Si trovano i due picchi coniugati nello spettro e si azzerano con un filtro notch. Molto meglio di un passa-basso, che sfoca tutto.
5. **Che condizione al bordo usa il filtraggio con la FFT?**
Periodica (wrap): la DFT tratta l'immagine come ripetuta.
6. **Perché in un banco di filtri serve la non linearità?**
Senza, le feature sono lineari nei pixel e il classificatore lineare non migliora (0.8127 contro 0.8129).
7. **Vantaggi e svantaggi del pooling?**
Tolleranza a piccoli spostamenti, meno feature; si perde la posizione. Troppo pooling fa crollare l'accuratezza.
8. **Cosa manca a un banco di filtri fissi rispetto a una CNN?**
Filtri imparati dai dati e più strati (gerarchia).

Cosa è verificato e cosa no

- **Rieseguito** con il codice dei notebook: tutte le celle di `demo.ipynb` (numeri stampati coincidenti con gli output salvati), le due soluzioni degli esercizi, il comportamento di `boundary="symm"`, la deviazione standard del kernel della maschera gaussiana. Tutti gli esperimenti di `filter_bank.ipynb` nelle tabelle con colonna "rieseguito", compreso il test finale. Script e risultati in `Lessons/L7/appunti_src/` (`verifica_demo.py`, `verifica_filter_bank.py`, `risultati_filter_bank.json`, `figs.py`).
- **Non rieseguito**: la tabella della challenge (output salvati del docente). Le soluzioni degli esercizi di `filter_bank` sono ricostruite da me: coincidono con gli output salvati per numero di feature, e per accuratezza entro 0.6 punti.
- **Ambiente**: numpy 2.5.3, scipy 1.18.1, scikit-image 0.26.0, scikit-learn 1.9.1, matplotlib 3.11.2.